

POSTED ON FEBRUARY 5, 2025 TO ML APPLICATIONS, SECURITY & PRIVACY

Revolutionizing software testing: Introducing LLM-powered bug catchers



By Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, Shubho Sengupta



WHAT IT IS

Meta's Automated Compliance Hardening (ACH) tool is a system for **mutation-guided, LLM-based test generation**. ACH hardens platforms against regressions by generating undetected faults (mutants) in source code that are specific to a given area of concern and using those same mutants to generate tests. When applied to privacy, for example, ACH automates the process of searching for privacy-related faults and preventing them from entering our systems in the future, ultimately hardening our code bases to reduce risk of any privacy regression.

ACH automatically generates unit tests that target a particular kind of fault. We describe the faults we care about to ACH in plain text. The description can be incomplete, and even self-contradictory, yet ACH still generates tests that it proves will catch bugs of the kind described.

Traditionally, automated test generation techniques sought merely to increase code coverage. As every tester knows, this is only part of the solution because increasing coverage doesn't necessarily find faults.

ACH is a radical departure from this tradition, because it targets specific faults, rather than uncovered code, although it often also increases coverage in the process of targeting faults. Furthermore, because ACH is founded on the principles of **Assured LLM-based Software Engineering**, it keeps verifiable assurances that its tests do catch the kind of faults described.

Our new research paper, "**Mutation-Guided LLM-based Test Generation at Meta**," gives details of the underlying scientific foundations for ACH and how we apply ACH to privacy testing, but this approach can be applied to any sort of regression testing.

HOW IT WORKS

Mutation testing, where faults (mutants) are deliberately introduced into source code (using version control to keep them away from production) to assess how well an existing testing framework can detect these changes, has been **researched for decades**. But, despite this, mutation testing has remained difficult to deploy.

In **earlier approaches**, mutants themselves would be automatically generated (most often using a rule-based approach). But this method would result in mutants that weren't particularly realistic in terms of how much of a concern they actually represent.

On top of that, even with the mutants being automatically generated, **humans would still have to manually write the tests that would kill the mutants** (catch the faults).

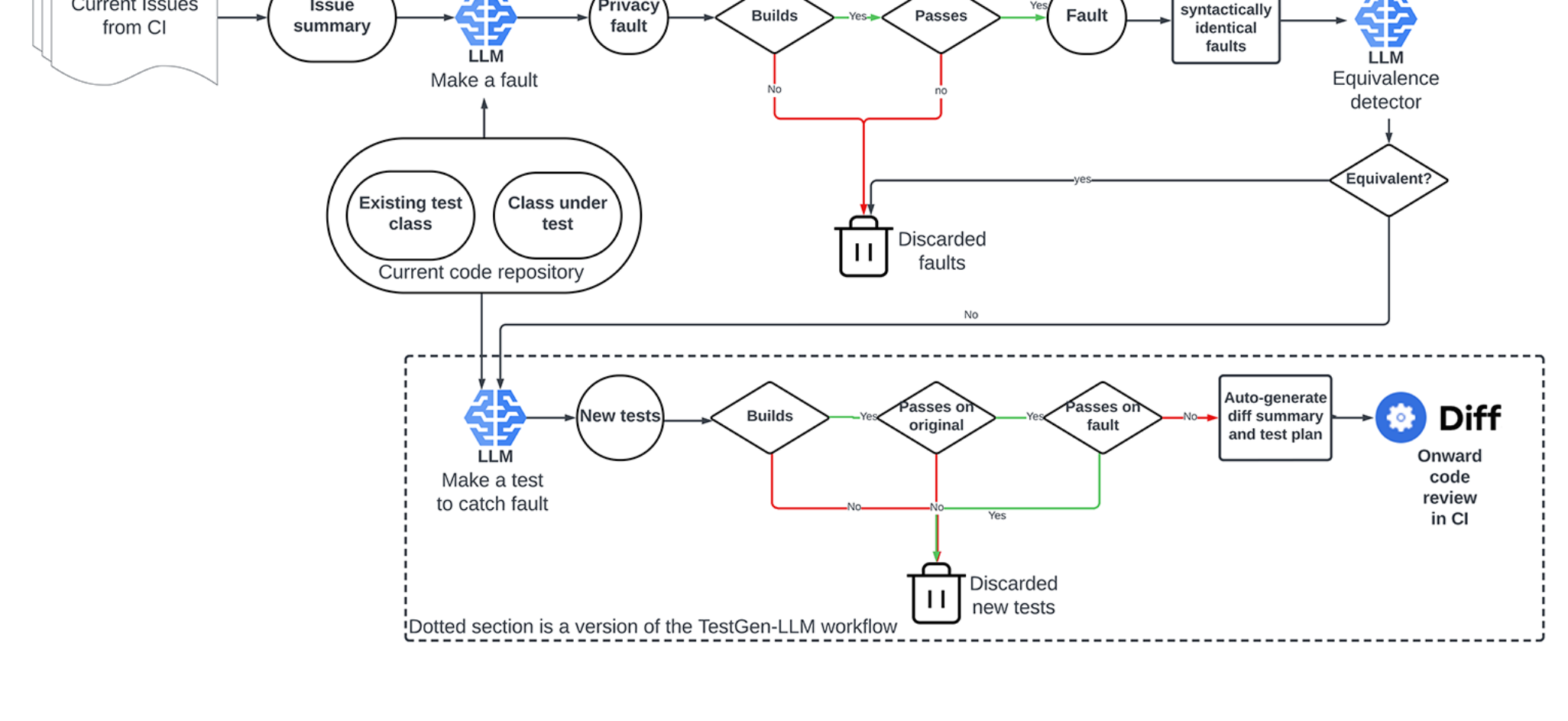
Writing these tests is a painstaking and laborious process. So engineers were faced with a two-pronged issue: Even after doing all of the work to write a test to catch a mutant, there was no guarantee the test would even catch the automatically-generated mutant.

By leveraging LLMs, we can generate mutants that represent realistic concerns and also save on human labor by generating tests to catch the faults automatically as well. ACH marries automated test generation techniques with the capabilities of large language models (LLMs) to generate mutants that are highly relevant to an area of testing concern as well as tests that are guaranteed to catch bugs that really matter.

Broadly, ACH works in three steps:

1. An engineer describes the kind of bugs they're concerned about.
2. ACH uses that description to automatically generate lots of bugs.
3. ACH uses the generated bugs to automatically generate lots of tests that catch them.

At Meta we've **applied ACH-assisted testing to several of our platforms**, including Facebook Feed, Instagram, Messenger, and WhatsApp. Based on our own testing, we've concluded that engineers found ACH useful for hardening code against specific concerns and found other benefits even when tests generated by ACH don't directly tackle a specific concern.



A top-level overview of the architecture of the ACH system. The system leverages LLMs to generate faults, check them against possible equivalents, and then generate tests to catch those faults.

WHY IT MATTERS

Meta has a very large number of data systems and uses **many different programming languages**, frameworks, and services to power our family of apps and products. But, how are our thousands of engineers across the world ensuring that their code is reliable and won't generate bugs that would negatively impact application performance, leading to privacy risk? The answer lies with LLMs.

LLM-based test generation and LLM-based mutant generation are not new, but this is the first time they've been combined and deployed in large-scaled industrial systems. Generating mutants and the tests to kill them have been traditionally difficult processes to scale. Since LLMs are probabilistic and don't need to rely on rigidly defined rules to make decisions, they allow us to tackle both sides of this equation – generating mutations and tests to kill them – very efficiently and with a high level of accuracy.

This new approach significantly modernizes this form of automated test generation and helps software engineers take in concerns from a variety of sources (previous faults, colleagues, user requirements, regulatory requirements, etc.) and efficiently convert them from freeform text into actionable tests – with the guarantee that the test will catch the fault they're looking for.

ACH can be applied to any class of faults and have a significant impact on hardening against future regressions and optimizing testing itself.

WHAT'S NEXT

Our novel approach combines LLM-based test generation and mutant generation to help automate complex technical organizational workflows in this space. This innovation has the potential to simplify risk assessments, reduce cognitive load for developers, and ultimately create a safer online ecosystem. We're committed to expanding deployment areas, developing methods to measure mutant relevance, and detecting existing faults to drive industry-wide adoption of automated test generation in compliance.

We will be sharing more developments and encourage you to watch this space.

READ THE PAPER

[Mutation-Guided LLM-based Test Generation at Meta](#)

Share this:

- Facebook
- Threads
- X
- LinkedIn
- Hacker News
- Email



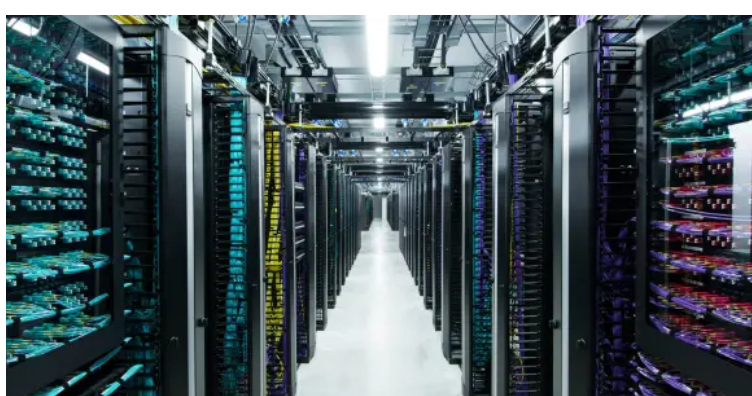
◀ Prev
Data logs: The latest evolution in Meta's access tools

Next ▶
Looking back at our Bug Bounty program in 2024



Read More in ML Applications

[View All ▶](#)



OCT 14, 2025
How Meta Is Leveraging AI To Improve the Quality of Scope 3 Emission Estimates for IT Hardware

OCT 13, 2025
OCP Summit 2025: The Open Future of Networking Hardware for AI

SEP 30, 2025
LLMs Are the Key to Mutation Testing and Better Compliance

SEP 29, 2025
Meta 3D AssetGen: Generating 3D Worlds With AI

SEP 29, 2025
Meta's Infrastructure Evolution and the Advent of AI

SEP 26, 2025
Networking at the Heart of AI — @Scale: Networking at Scale Recap

Available Positions

Software Engineer, Android

LONDON, UK

Software Engineer - Computer Graphics (Technical Leadership)

PITTSBURGH, US

Software Engineer - Computer Graphics (Technical Leadership)

REDMOND, US

Software Engineer - Computer Graphics (Technical Leadership)

BURLINGAME, US

Software Engineer, Infrastructure

SUNNYVALE, US

[See All Jobs](#)

Technology at Meta

Engineering at Meta - X

[Follow](#)

AI at Meta

[Read](#)

Meta Quest Blog

[Read](#)

Meta for Developers

[Read](#)

Meta Bug Bounty

[Learn more](#)

RSS

[Subscribe](#)

Open Source

Meta believes in building community through open source technology. Explore our latest projects in Artificial Intelligence, Data Infrastructure, Development Tools, Front End, Languages, Platforms, Security, Virtual Reality, and more.

ANDROID IOS WEB BACKEND HARDWARE

[Learn More](#)